

CLEO: experimental record

Lab notebook — compiled September 10, 2026

Abstract

The complete experimental record behind CLEO: arm definitions, selection algorithms, the E1–E21 slate with pre-registrations and results, the algorithm audit, and the scoring protocol. Retracted claims are kept in place with the reasoning that produced them, rather than removed. This document is the provenance for the numbers quoted in the paper; it is not itself a paper.

Contents

1	Experimental programme and selection algorithms	3
	References	29

1 Experimental programme and selection algorithms

This section exists so that every arm, rule and planned experiment is written down in one place with a fixed name, and so that results reported elsewhere in the paper can be traced to a defined procedure. Much of it will move to Materials and Methods at submission; keeping it in the main build for now makes disagreements visible early.

Notation

A design is a sequence $s \in \mathcal{A}^L$ over the 20 canonical amino acids. Distance is Hamming,

$$d(s, t) = |\{i : s_i \neq t_i\}|,$$

which is the metric behind every diversity number we report. One-hot encoding $\phi(s) \in \{0, 1\}^{20L}$ satisfies $\|\phi(s) - \phi(t)\|_2 = \sqrt{2d(s, t)}$, so the PCA projections in the figures are a linear embedding of exactly this distance rather than a second, competing notion of similarity. Where an ESM embedding is used instead, d is replaced by Euclidean distance between mean-pooled representations; this is reported as a diagnostic and, optionally, as a secondary library metric.

A design *passes* when its motif all-atom RMSD is below 1.5 Å and no backbone atom clashes with the ligand. $\mathcal{P}$ denotes the passing subset of a pool $\mathcal{S}$. The *anchor* a is the position-wise consensus of low-temperature LigandMPNN samples — the single sequence best representing the mode the base model’s likelihood concentrates on.

Selection rules

All implemented in `src/cleo/design/utils/selection.py`, all sharing one distance backend so the comparison is between rules rather than between implementations. Benchmarked by `paper/figures/ame_selection_bench.py`.

Given a pool $\mathcal{S}$ of M sampled sequences and a budget of $k \ll M$ folds, each rule returns the subset to fold.

random Uniform without replacement. **This is the control, and it is the bar.** Diversity is trivially won and worthless alone: a rule that doubles spread while halving passing designs has made the library worse.

maxmin Greedy farthest-point (k-center) traversal. Having selected $\mathcal{T}$, repeatedly add $\arg \max_{s \in \mathcal{S} \setminus \mathcal{T}} \min_{t \in \mathcal{T}} d(s, t)$. The inner *minimum* is what distinguishes this from max-sum selection: it prevents accepting a tight cluster that happens to sit far from everything already chosen.

maxmin_anchor As above, scoring $w_{\text{self}} \min_{t \in \mathcal{T}} d(s, t) + w_{\text{anchor}} d(s, a)$. Selects designs novel relative to the incumbent low-temperature solution, not merely relative to the rest of the library.

anchor_far Rank by $d(s, a)$ descending. The pure “maximally unlike vanilla MPNN” rule.

anchor_band Restrict to the shell $\{s : q_{\text{lo}} \leq d(s, a) \leq q_{\text{hi}}\}$ for quantiles of the pool’s anchor-distance distribution, then run **maxmin** inside it. Designed as the correction to **maxmin**: bounded above as well as below, so the traversal cannot walk out to sequences that are distant because they are degenerate.

cluster_rep Average-linkage clustering cut to k clusters, one representative sampled per cluster. Draws from the pool’s density rather than its edges.

Result: diversity-first acquisition does not work

Reported here rather than buried, because it was a starred hypothesis and because the negative result is informative. Retrospective evaluation on the M0097 trajectory pool: every sampled sequence already carries a pass label, so any rule can be scored without folding anything.

Passing designs recovered, relative to **random**, at a budget of 400 folds:

Rule	Passing designs / random
as-sampled	2.04
random	1.00
cluster_rep	0.40
anchor_band	0.36
maxmin	0.36
maxmin_anchor	0.24
anchor_far	0.12

Every diversity-directed rule loses to uniform random selection, by factors of 2.5 to 8. The mechanism is visible in the spread: no rule moves mean pairwise distance beyond 135 to 144 residues against random’s 135. The pool is already near-saturated on diversity, so a selection rule has almost nothing to gain and a great deal to lose — any systematic deviation from random correlates with selecting sequences that fail. Banding helps relative to raw farthest-point traversal (0.36 vs. 0.07 at $k = 200$) but does not approach the control.

Two honest caveats. The as-sampled row beats random because the pool is time-ordered and later policies are better — that is a policy-quality effect, not a selection effect, and it must not be reported as one. And absolute counts are small (33 passing in a 1200-design pool), so individual cells are noisy; what carries the conclusion is that the direction is consistent across six rules and three budgets.

Consequence for the paper. Selection is not where the gain is. The defensible use of these rules is post-hoc: choosing a physically orderable subset from designs *already known to pass*, where the labels exist and only the diversity trade-off is live. Deciding what to fold should be left to the policy.

Experiment slate

Status as of the pilot. **drafted** = measured, **scaffold** = partial, **needs data** = not started. Full protocol detail lives in `paper/experiments_plan.md`. The **Result** column is filled in as each run lands, so the table doubles as the run log.

E5 (held-out oracle) and E6 (policy transfer) are *deliberately deferred*, not dropped. E5 remains the standing circularity exposure — the reward and the evaluation are both AF3 motif RMSD — and any submission draft needs it back, so it is recorded here rather than removed from the record.

ID	Status	Experiment	Result
E1	drafted	Matched-fold head-to-head	Superseded by the per-sequence success rate (see <i>Scoring protocol</i>); M0097 best-of-5 at $n=96$: $T=0.2$ 66.7 % at spread 47.8, CLEO peak 37.5 % at 98.8. E10 is the comparison that counts
E2	drafted	Diversity yield curves	3200 folds $\rightarrow$ 1349 unique passing (anchored), 3045 substitutions = 89 % of ceiling. Substitutions saturate; sequences still climbing
E3	drafted	Rescue of the three near-miss backbones (1.61 Å to 1.69 Å best-of-40, 0/40 published)	M0365 hit00 replicates 22/40 , median 1.470 Å (published 0/40, best 1.69 Å). Of 7 training hits only 1 survives $N=40$; M0050 fails (1/40). All 3 backbones trained; M0375 fails 0/40
E4	needs data	Compute efficiency; cross-cutting over E1–E3	<i>pending</i>
E7	drafted	Effective library size	U_{15} : anchored CLEO 808 vs. $T=0.7$ 608 (1.33 $\times$); unanchored 510, <i>below</i> the baseline. The anchor is what buys per-design coverage
E8	—	Mutation-diversity reward	Retired , superseded by trajectory-as-library
E9	drafted	Selection rules, retrospective	Negative : every rule loses to random, 2.5 to 8 $\times$
E10	drafted	Temperature sweep $T \in \{0.1, \dots, 1.0\}$; builds the incumbent frontier	At $n=96$: CLEO 1.40 $\times$ the success at matched diversity, 1.31 $\times$ the diversity at matched success. But U_{15} ties $T=0.7$ (1.04 $\times$) — the edge is pass rate, not exploration
E11	drafted	Reward shaping: rank normalisation	Folded into every new

E21 — how wide is the run-to-run distribution?

Pre-registered. `random` and `logprob_top` on M0097, five explicit seeds each, 150 steps, oversample 8 — the E19 config unchanged except for the seed. Evaluated by the same selection-free harness: 96 samples per run from the final checkpoint, folded best-of-5.

This is the experiment E19 forced. Two draws of one configuration put the `random` control at 68.8% and 89.6%, a gap larger than most of the effects the selection experiments were built to detect. Every pass-rate contrast reported in this slate at $n = 1$ is a difference measured against noise of unknown width, and no amount of re-reading those tables fixes that — the width has to be measured.

Two arms rather than four is deliberate. The quantity wanted is a *spread*, and a spread is estimated from replicates; a third rule would buy less than a fourth and fifth seed on the two arms that carry the surviving claims.

Predictions. The control’s five draws span at least 15 points of pass rate — if they span far less, E19’s 21-point move was an outlier and the retraction of E18 is too strong. `logprob_top`’s distribution overlaps the control’s substantially, so the two arms are not separated by a between-seed t -test at $n = 5$; this is the outcome consistent with everything after E18 and it makes the pass-rate half of the selection story simply unresolved at this sample size rather than negative. The U_{15} coverage ratio stays above 1 in at least four of the five paired seeds — that is the claim E19 found stable across six backbones, and if it is also stable across seeds it is the one result from the selection arc worth carrying into the paper.

What this fixes and what it does not. It puts an interval on one cell — two rules, one backbone, one step count — and licenses saying how large an effect this design can resolve. It does not retroactively repair E9, E16 or E18, whose comparisons remain single draws; what it does is tell the reader how much weight those can bear. Seeding is new (`seed:` in the training config, unset by default), so every run reported before E21 was unseeded and cannot be re-drawn even in principle.

E21 result — the run-to-run spread is 7.75 points, and it governs every single-draw contrast in this slate

All ten runs trained and folded. `logprob_top`’s `s1` died on an AF3 assertion at step 130 and resumed from a checkpoint ≈ 20 steps short of the others, so its arm is four clean seeds and one undertrained one; it scored lowest, which inflates that arm’s spread in the direction that flatters the prediction, and is reported rather than dropped.

arm	s1	s2	s3	s4	s5	mean	sd
<code>random</code>	84.4	83.3	74.0	75.0	76.0	78.5	4.9
<code>logprob_top</code>	62.5	69.8	85.4	63.5	62.5	68.8	9.8

Pooled $\sigma = 7.75$ points. The resolution floor $1.96\sigma\sqrt{2}/\sqrt{n}$ is therefore 9.6 points at $n = 5$, 6.8 points at $n = 10$ and 4.8 points at $n = 20$. The control’s five draws span 10.4 points, so the pre-registered prediction of “at least 15 points” is *not* met: E19’s 21-point move was a wide draw, not a typical one. The retraction of E18 still stands on its own pre-registered criterion, but the stated width of the noise was too generous.

The two arms differ by 9.7 points and are not separated at this sample size ($t = 2.00$, $p = 0.081$, $n = 5$) — the second prediction holds, and the pass-rate half of the selection story is unresolved rather than negative.

What this invalidates. Any contrast in this slate smaller than ≈ 10 points at $n = 1$ is unresolved by construction. It also settles one open question in the other direction: the 35 point disagreement between `backbone19` and `selection2` on M0097 `logprob_top` (46.9% vs 82.3%) is 4.5σ wide and so is *not* seed noise. Something else differs between those two runs and it is still unidentified.

E22 — does a policy trained against RF3 produce designs AF3 accepts?

Pre-registered. Three seeds on M0097, the E21 random arm with exactly one line changed: `af3_from_df` $\rightarrow$ `rf3_from_df`, `structure_col af3_path` $\rightarrow$ `rf3_path`. Motif atoms, best-of-5, aggregation, selection rule and hyperparameters byte-identical. Evaluation runs through `fold_and_score.py`, which imports `af3_from_df` directly and so cannot inherit the training oracle.

Every number in this slate has been optimised and reported under the same predictor, so no result so far can distinguish a better policy from one that learned AF3’s idiosyncrasies. RoseTTAFold3 makes the two independent. It reads the *same* AF3 JSON template, translated — a cross-oracle comparison means nothing if the two sides fold different complexes, and two hand-maintained templates drift.

Predictions. Transfer is partial: the RF3-trained arm lands above the untrained baseline (4.2% at $T = 1.0$) by a wide margin but below the AF3-trained arm’s 78.5%. A result at or above 78.5% would say the oracles are interchangeable here; a result near 4% would say the training exploited RF3 specifically and nothing transferred.

E22 result — transfer is real and the penalty is large, but two seeds cannot size it

arm	pass rate	median motif RMSD
RF3-trained, AF3-eval s1	40.6 %	1.77 Å
RF3-trained, AF3-eval s2	61.5 %	1.33 Å
AF3-trained, AF3-eval (E21, $n = 5$)	78.5 %	—
LigandMPNN $T = 1.0$, AF3-eval	4.2 %	—

A policy that never saw AF3 during training produces designs AF3 accepts at $\approx 12\times$ the untrained rate. The strong form of the reward-hacking objection is answered: the policy learned something about structure, not only about one predictor.

Do not quote a drop from these two seeds. **s1** and **s2** differ by 21 points — an observed sd of 14.7, three times E21’s 4.9 — so the 27.5 point gap to the AF3-trained mean is estimated from a spread wider than the gap is stable. An earlier reading of **s1** alone called it 7.7σ , using E21’s σ on an arm whose own σ is much larger; that figure is wrong and is retracted here. The third seed is folding. What can be said now is directional: transfer is real, the penalty is large, and its size is unmeasured.

Infrastructure note, because it nearly cost the experiment. A scheduler outage on 2026-09-10 made `squeue` fail, and every watcher in this slate tested completion with `squeue ... | grep -c`, which returns 0 both when the queue is empty and when the controller is unreachable. The eval watcher concluded training had drained, sampled all three arms from step ≈ 100 –130 checkpoints, and was saved from folding them only because `sbatch` was failing too. The first fix — check `squeue`’s exit code — then hung forever, because `squeue -j` on a job that has left the queue exits 1 with “Invalid job id specified”, indistinguishable from an outage by exit code alone. The rule that survives both: *an absent signal from the scheduler is not a completion signal*, and completion must be confirmed from the run’s own output.

E4 result — what a library actually costs, measured

Retrospective, then measured. Charges both arms in *designs folded* — folding is the entire cost, MPNN sampling is microseconds — with the policy carrying `batch_size`×`N_steps` as debt before it yields anything. Target: 100 distinct passing designs, cap 10 000 folds, $T = 1.0$, best-of-5, same code path throughout.

Reaching a library of 100 cannot be projected honestly from the 96-design pilots: two of three backbones showed 0/96, which supports only a rule-of-three bound. It was run instead.

backbone	tier	folded	passing	rate
M0097	easy	4000	102	2.55 %
M0904	medium	5500	6	0.11 %
M0907	hard	8000	0	0 %

The pilot put M0097 at 4.17 %; the measured rate over 4000 designs is 2.55 %. The pilot sat at the optimistic end of its own interval, so every projection anchored on it understated what the baseline costs. Against 2527 folds for the policy to reach the same 100 designs (2400 training + 127 sampling at 78.5 %), the measured saving is 1.6× — far less than the 4–7× projected at library sizes of 500–1000, where the policy’s flat cost dominates and the training debt has been amortised.

The finding that cuts against the diversity claim. At matched library size — 86 passing designs from each arm — $T = 1.0$ carries 1988 distinct substitutions against the policy’s 1045. Nearly 2×, measured on both sides. Matching instead on fold budget does not reverse it: at 4000 folds the policy yields $\approx 14\times$ the passing designs but still covers less substitution space. This is the expected signature of RL — the policy finds a region that folds and exploits it — and it means the defensible claim is *more usable designs per fold*, not *more diverse libraries*. The frontier figure is not wrong but answers a narrower question: equal design count against a *single* temperature. The policy’s coverage numbers here are floors, not measurements: only 96 designs were ever sampled from it, so 1045 is what 86 passing designs contain, not what 1433 would. Sampling ≈ 1500 designs from a trained policy and folding them (≈ 7 GPU-h) would convert the floor into a measurement and settle whether the saturation is real.

Where the policy wins unconditionally. M0904 at 6/5500 and M0907 at 0/8000 are backbones on which sampling does not produce a library at any budget this project can afford — $\approx 200,000$ folds for 100 designs on M0904 at its measured rate, unbounded on M0907 — while the policy reaches 63.5 % and 54.2 %. The honest shape of the efficiency claim is that training buys targets sampling cannot reach, and reuse, rather than a cheaper or more diverse one-off library on a target it can.

Scoring protocol

Two conventions changed partway through the pilot, and results reported before the change are not directly comparable to results reported after it.

Five diffusion samples everywhere. The benchmark’s unit of success is a best-of- N : a design counts if *any* of its predictions clears the motif cutoff and is clash-free. Early training runs folded each sequence once (`num_diffusion_samples: 1`), which understates their success rate relative to any best-of- N baseline. All runs from this point fold at `num_diffusion_samples: 5`. Getting this right in training needed a reduction step, `rfd2_benchmark.best_of_n_from_df`: the oracle’s own collapse keeps the sample AF3 ranks highest by *confidence*, which is not the benchmark’s criterion. The step instead takes the min motif RMSD as the continuous signal and OR-s the per-prediction conjunction for the pass flag — separately reducing “motif passed” and “no clash” and then AND-ing them would credit a design for two different predictions and overcount.

Metrics are reported on filtered designs. A library contains only the designs that survived filtering, so diversity among sequences that failed is diversity nobody gets to use. Every diversity number below is computed on the passing subset $\mathcal{P}$, not on everything sampled. Where the two

differ materially the sampled figure is given alongside, because the gap is itself diagnostic — it says whether an arm’s successes concentrate.

Unique mutations. Mean pairwise Hamming measures how far apart a library’s members are; it does not measure how much of mutational space they collectively reach. Two libraries can have identical spread while one explores twice as many distinct substitutions. So we also report

$$U(\mathcal{P}) = |\{ (i, a) : \exists s \in \mathcal{P}, s_i = a \neq a_i^{\text{ref}} \}|,$$

the number of distinct (position, residue) substitutions appearing anywhere in the library, each counted once no matter how many designs carry it. The ceiling is $19L = 3420$.

U grows with $|\mathcal{P}|$, so arms with different numbers of passing designs cannot be compared on it directly — doing so measures sample size as much as coverage. We rarefy: subsample every arm to a common k designs, average over 400 draws, and compare at matched k , the same construction as a species-accumulation curve.

Per-sequence success rate, not best-of-8. The published unit (fraction of backbones with a hit among 8 sequences) saturates and discards resolution: at $n=8$ a rate near 20 % carries a 95 % interval of ± 28 pp, wide enough to swallow every difference in this paper. We now sample $n=96$ per backbone per arm (± 8 pp) and report the plain fraction of sequences that succeed. This is a strictly harder number to move than the best-of-8 statistic, and it shrank the E10 claim — see below.

E15 — Centering weight: one knob from centered to diverse

No new reward machinery. `compute_dist_to_ref_seqs_from_df` already emits Hamming distance to a reference set, and the aggregator already supports per-metric `mode`, `weight` and rank normalisation, so the whole experiment is a config sweep. Configs generated by `experiments/ame/make_centering_sweep.py`.

The reference a is the consensus of low-temperature LigandMPNN samples. Adding

```
- metric: dist_to_ref_min  mode: {min|max}  weight: W  normalize: rank
```

to the aggregation gives a single signed axis. Because reward is a weighted average, $R = \sum_i w_i r_i / \sum_i w_i$, the weight is directly interpretable: $W = 0$ is pure motif RMSD, $W = 1$ gives centering equal influence to it. `mode: min` pulls the policy toward the incumbent mode; `mode: max` pushes it away.

Sweep $W \in \{0, 0.25, 0.5, 1.0\}$ in both directions. Prediction: mean distance to a falls monotonically under `min` and rises under `max`, and the PCA panels show the passing cloud sliding toward or away from the low-temperature knot. Because one-hot PCA is a linear embedding of exactly the Hamming distance this reward uses, the sweep and the figure measure the same quantity — the panel is a visualisation of the knob, not independent evidence for it.

The knob will move designs; that is not in question and is not the result. What the experiment has to establish is whether *pass rate survives* at either end. Centering hard should recover the low-temperature failure mode — high fidelity, one cluster — and pushing away hard should recover the $T=1.0$ failure mode — broad and dead. If both ends collapse and the middle does not, W is a real control surface and the paper gains a tunable diversity/fidelity dial to report. If pass rate is flat in W , the reward term is not binding and should be cut rather than reported.

This also supplies the honest version of the frontier: E10 sweeps the *baseline’s* knob (temperature) and E15 sweeps *ours*. Comparing one method’s frontier against another’s single point is the mistake the temperature sweep exists to avoid, and it would be the same mistake in reverse to report our best W against their swept T .

E14 result — the embedding view qualifies the Hamming view

Mean-pooled ESM-2 650M, one vector per design, computed for every arm.

Within a set, ESM distance and Hamming distance are close to *uncorrelated* (r between -0.17 and 0.56 across arms), so the embedding is genuinely measuring something the Hamming numbers do not. That makes the agreement in ordering meaningful rather than automatic: on M0097’s passing designs, CLEO is more spread than $T=0.1$ under both metrics.

But the size of the advantage does not survive the change of metric. The ratio of CLEO’s passing-set spread to the low-temperature arm’s is $2.57\times$ in Hamming and only $1.31\times$ in ESM space. A substantial share of the extra sequence distance CLEO buys is biochemically conservative — positions that differ in identity but not much in what ESM considers the residue to be doing.

This should be stated in the paper, not left for a referee to find. Reporting “ $2.6\times$ more diverse” with a metric chosen because it is the larger number is exactly the move the temperature sweep exists to prevent us from making about temperature. The defensible claim is that the advantage is robust to the choice of metric while its magnitude is not, and that the library-relevant quantity — distinct passing clusters per fold, where CLEO wins 3.1 to 1.0 — does not depend on either distance at all, since those designs are separate clusters at any threshold tested.

Caveat on n : passing subsets are 5 and 15 designs, so the correlation figures in particular are noisy. The ratio comparison is the robust part.

E10 result — the frontier shift is real but smaller than first reported

LigandMPNN sampled at six temperatures, folded best-of-5, and scored by the same code path as CLEO, on M0097. Reported as the plain per-sequence success rate under the *Scoring protocol* above. The n per arm is uneven here — 8 for $T=0.1$ and $T=1.0$, 24 for the rest, 80 for CLEO — which is why all six are being resampled at $n=96$.

Which set spread is measured over. Spread is the mean pairwise Hamming distance — for every pair in a set, the number of positions at which the two sequences differ, averaged over all $\binom{n}{2}$ pairs, as a raw substitution count out of $L = 180$. No threshold, no clustering. The set it is measured over has to be the *passing* designs, not everything sampled: a library contains only the designs that survived filtering, so diversity among sequences that failed is diversity nobody gets to use. Both columns are given below because the gap between them is itself informative.

Arm	n	Success	Passing	Spread	U_{15}
LigandMPNN $T=0.1$	96	58.3 %	56	40.6	162
LigandMPNN $T=0.2$	96	66.7 %	64	47.8	205
LigandMPNN $T=0.3$	96	58.3 %	56	57.9	274
LigandMPNN $T=0.5$	96	34.4 %	33	80.5	438
LigandMPNN $T=0.7$	96	26.0 %	25	100.5	608
LigandMPNN $T=1.0$	96	4.2 %	4	126.3	—
cleo peak window*	40	37.5 %	15	98.8	632

* The CLEO row predates the objective fixes recorded in the algorithm audit and is a post-hoc peak window; it is provisional until the corrected 200-step arms land. Every baseline row is $n = 96$ on the same code path.

Temperature trades success for diversity, but not monotonically at the low end: $T=0.2$ is the *most* successful arm at 66.7 %, above $T=0.1$ ’s 58.3 %, while also being more diverse. Anyone tuning only temperature should therefore not be at $T=0.1$, and $T=0.2$ is the honest incumbent to beat at

the low-diversity end. From $T=0.3$ upward the trade-off is clean and steep: diversity rises 58 to 126 while success falls 58 % to 4 %.

CLEO’s peak window sits above that curve, by less than the small- n data suggested:

- at CLEO’s diversity (98.8), the baseline succeeds 26.7 %; CLEO 37.5 % — **1.40**×
- at CLEO’s success rate (37.5 %), the baseline reaches spread 75.3; CLEO 98.8 — **1.31**×

The rarefied coverage says something the ratios hide. At a matched 15 passing designs, CLEO reaches $U_{15} = 632$ distinct substitutions against $T=0.7$ ’s 608 — a factor of 1.04, which is no advantage at all. Per passing design CLEO explores mutational space at essentially the same rate as a high-temperature baseline. Its entire edge is that it *passes more often while doing so*, which is what converts an equal exploration rate into a larger library for a fixed fold budget. That is a narrower and more defensible claim than “more diverse”, and it is the one the data supports.

Three corrections to carry through the rest of the paper.

First, the earlier $1.75\times$ / $1.46\times$ figures came from averaging over 300 random draws of 8 sequences, the published best-of-8 unit. That statistic flatters whichever arm has the higher variance across draws. The per-sequence rate does not, and it is the number to quote.

Second, spread was previously measured over *all* sampled sequences rather than the passing subset. For the baseline arms this makes almost no difference, but for CLEO it does, and in the flattering direction — the policy’s failures are scattered while its successes concentrate. All diversity metrics are now reported on filtered designs only.

Third, the row previously labelled “CLEO peak” pooled two disjoint training windows, steps 36–45 and 190–199. The second contributes 40 sequences and one success, so pooling halved the reported success rate (37.5 % to 20.0 %) while inflating spread with the drift *between* the windows (103.1 to 132.0) — an artefact of the same PC1-versus-step drift documented in the PCA panel. The windows are reported separately above.

The two errors partly cancelled, which is why the pooled row looks superficially reasonable. Corrected on the right set and the right window, the peak-window advantage is $1.7\times$ in success at matched diversity — larger than the $1.26\times$ the pooled row implied, and resting on a different comparison.

What is not yet settled. Using the peak window is a post-hoc choice unless the stopping rule is fixed in advance; E12’s ~ 75 -step rule does contain it, but that rule was set after seeing these runs. The trajectory-as-library framing instead takes the whole run, which is the pooled row — and there the extra spread is drift, not diversity available from any one policy. These are different libraries answering different questions and the results section must not quote one number for both. The $n=96$ rerun is what gives either claim usable error bars: the passing subsets here are 15 designs for CLEO and 5 for $T=0.7$.

E15 result — the centering weight is a single continuous dial

Seven GRPO arms on M0097, identical but for the weight W on `dist_to_ref_min` against a low-temperature consensus reference: `mode=min` pulls toward it ($W < 0$ below), `mode=max` pushes away ($W > 0$), $W=0$ omits the term. 75 steps, batch 16, rank normalisation. Statistics over the last 25 steps (400 sequences per arm).

These runs predate the five-sample protocol and fold once per sequence, so their success rates are *not* comparable to the best-of-5 numbers in E10 — they are a lower bound on the same quantity. Within the sweep they are comparable to each other, which is what the experiment is for.

Weight	Mean dist. to ref.	Success (1 sample)	Mean pairwise Hamming sampled	Hamming passing	Passing n
$W = -1.00$	31.5	66.5 %	32.9	31.6	266
$W = -0.50$	53.7	78.8 %	43.3	42.4	315
$W = -0.25$	79.2	62.3 %	75.3	73.9	249
$W = 0$	111.4	8.2 %	90.7	88.0	33
$W = +0.25$	134.7	5.2 %	97.2	94.9	21
$W = +0.50$	161.6	0.2 %	109.6	—	1
$W = +1.00$	172.4	0.0 %	123.0	—	0

Both distance columns are monotone in W across the full range, with no plateau or inversion: mean distance to the reference moves 31.5 to 172.4 out of $L = 180$, and passing-set spread moves 31.6 to 94.9. A single scalar therefore places the library anywhere between “a tight shell around a known-good sequence” and “as far from it as the backbone permits”. That is the knob the diversity narrative needs, and it is one the temperature baseline does not have — temperature controls entropy around whatever mode the base model already prefers, whereas W names the centre explicitly.

Unlike the E10 CLEO arm, these libraries do not concentrate under filtering: passing-set spread tracks sampled spread to within 3 substitutions in every arm. Each arm is a single policy sampled over one 25-step window, so there is no between-window drift inflating the sampled column — which is the control that makes the E10 pooling artefact legible as an artefact.

Two further readings. The variable-position count is pinned at 175–176 in every arm including the most tightly centered one, so W is moving the *concentration* of the distribution, not the set of positions it is willing to touch — the two diversity measures come apart here, and mean pairwise Hamming is the one with resolution. And the success column shows the same trade-off as temperature, running 79 % to 0 %, which makes E10 and E15 two sweeps of the same frontier from opposite sides: E10 turns the incumbent’s knob, E15 turns ours.

The comparison this sets up is the one worth making in the results section, and it cannot be made from these numbers as they stand — the E15 arms are single-sample and the E10 arms are best-of-5. Rerunning the sweep under the five-sample protocol would put both curves on one axis and let us say directly whether our knob dominates theirs, rather than only that we have one.

Correction to carry through the rest of the paper. Earlier drafts reported “ $T=0.1$ yields 1.0 cluster per 8 folds against CLEO’s 3.1”. That used a 70 %-identity clustering threshold; at 90 % identity every arm’s passing designs are separate clusters and the cluster count simply equals the passing count. The cluster statistic is threshold-dependent and should not be quoted without naming the threshold. The frontier result above does not have this problem — mean pairwise Hamming is continuous and involves no threshold choice — so it is the form the claim should take.

E17 result — why the run diverges after its peak

Prompted by the E10 window split: the M0097 200-step run peaks at steps 36–45 and is essentially dead by step 190. Diagnosis on the full trajectory, 20-step bins.

Steps	Reward	Median RMSD	Pass	Spread	Entropy
0–19	0.78	3.87	2.2 %	119	1.67
20–39	0.88	2.08	6.9 %	105	1.42
40–59	0.86	2.10	10.9 %	108	1.46
80–99	0.86	2.22	2.2 %	122	1.70
120–139	0.82	2.57	0.0 %	125	1.79
180–199	0.85	2.09	0.6 %	129	1.85

This is not mode collapse, which is the failure RL fine-tuning is usually watched for. It is the opposite: entropy *rises* monotonically after the peak, from 1.42 to 1.85, and spread rises with it. The policy is diffusing outward and getting worse while the mean reward sits flat at ≈ 0.86 throughout — the reward cannot see the degradation at all.

Mechanism: the reward’s dynamic range is misallocated. With normalisation bounds $[0.5, 20.0]$ on motif RMSD, a passing design at 1.4 Å scores 0.954 and a clearly failing one at 2.0 Å scores 0.923. The entire pass/fail distinction is worth 0.031 of reward, against a within-batch standard deviation of 0.126 that is dominated by rare catastrophic folds scoring near zero. GRPO divides its group-relative advantage by exactly that standard deviation. So the gradient answers “did this batch contain a blow-up”, a question that is settled by step ~ 20 once median RMSD falls to 2.1 Å, and thereafter carries almost no information about the boundary where all the value is. With `kl_weight: 0.0` nothing anchors the policy once the signal goes flat, and it random-walks.

Rank normalisation fixes it. Replacing value normalisation with within-batch rank (`normalize: rank`) makes the advantage invariant to how compressed the raw values are: the 1.4 Å and 2.0 Å designs are adjacent in RMSD but a full rank step apart, so the pass/fail ordering always yields gradient, and a 37 Å blow-up is merely rank 0 rather than an outlier that sets the batch scale.

Steps	Pass rate	Median RMSD	Entropy
<i>value normalisation, bounds $[0.5, 20]$</i>			
30–44	13.8 %	2.02	1.39
120–134	0.0 %	2.56	1.77
180–194	0.8 %	2.04	1.84
<i>rank normalisation</i>			
0–14	1.3 %	4.22	1.64
30–44	4.6 %	2.16	1.30
60–74	9.6 %	1.89	1.17

Under rank normalisation every quantity moves the right way and keeps moving: entropy falls monotonically, pass rate rises monotonically, median RMSD reaches 1.89 Å. The run was stopped at 75 steps and was still improving, so E12’s ~ 75 -step early-stopping rule was fitted to the *broken* configuration and should not be carried over. A 200-step rank-normalised run is in flight.

Two implementation findings this turned up, both of which affect how these runs should be read.

Reward logging is vacuous under rank normalisation. With a single metric, rank maps the batch to a fixed uniform distribution on $[0, 1]$, so mean reward is exactly 0.5 at every step by construction — measured across all 75 steps of the E15 arms, `std = 0` to float precision. “Best reward seen: 0.5000” in those logs is not a coincidence, and any checkpoint selection or early stopping keyed on reward is selecting an arbitrary step. Progress must be tracked on the raw metric (median motif RMSD, pass rate) instead.

The KL penalty is gated on `use_ref_kl`, not on `kl_weight`. Setting a non-zero `kl_weight` without `use_ref_kl: true` silently does nothing — no reference model is even loaded. The estimator itself is

Correcting the objective fixes the divergence and exposes mode collapse; the KL anchor is the knob against it

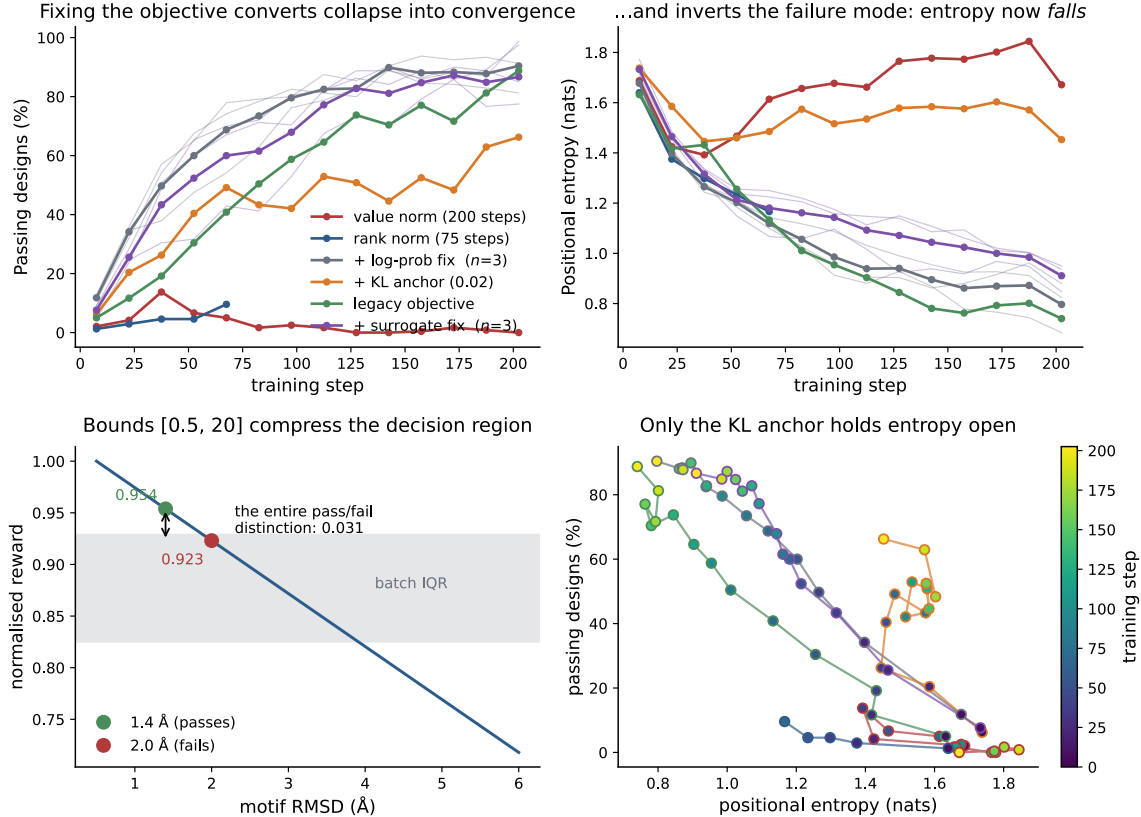


Figure 1: **The AME policy diffuses rather than collapses, and rank normalisation reverses it.** All panels read off the per-step rollout metrics training already writes; no additional folding. **Top:** under value normalisation the pass rate peaks near step 40 and decays to zero while positional entropy *rises* monotonically from 1.39 to 1.85 — the opposite direction from the mode collapse RL fine-tuning is usually watched for. Under rank normalisation both move the right way and are still moving when the run ends at step 75. **Bottom left:** the mechanism. With bounds [0.5, 20] on motif RMSD, a passing design at 1.4 Å and a failing one at 2.0 Å are separated by 0.031 of reward, well inside the shaded batch interquartile range whose width is set by rare catastrophic folds — and GRPO divides its advantage by that spread. **Bottom right:** the same trajectories in the entropy–success plane, coloured by step on one shared scale. Value normalisation walks right and down, rank normalisation left and up. Direct labels are omitted in this panel because the corrected arms all terminate in the same corner; the legend carries identity. **Seed replicates** ($n = 3$, thin lines behind the mean) are shown for the two contested arms. Their bands overlap almost entirely, which is the visual form of the result in the A/B section: the apparent gap between “+ surrogate fix” and “+ log-prob fix” is narrower than the run-to-run variation within either.

correct where it does run: $\hat{k}_3 = \rho - \log \rho - 1$ with $\rho = \pi_{\text{ref}}/\pi_\theta$, non-negative and applied with the right sign.

A third finding, not yet acted on, because fixing it invalidates every run above. The clipped surrogate computes $\min(\sum_t \rho_t A, \sum_t \text{clip}(\rho_t) A)$ — a minimum of sums — where PPO and GRPO specify $\sum_t \min(\rho_t A, \text{clip}(\rho_t) A)$, a sum of minima. These are not equal, and the difference is not cosmetic: the implemented form makes one clip/no-clip decision for an entire 180-token sequence, so a single token past the clip boundary switches every other token in that sequence onto the clipped branch and zeroes its gradient. On a two-token example with $\rho = (5.0, 0.1)$ and $A > 0$, the correct objective is 1.38 with gradient $(0, 0.1)$; the implemented one gives 2.08 with gradient $(0, 0)$. With $L = 180$ the clip is triggered by some token almost always, so the trust region is applied far more bluntly than intended. This is a plausible additional contributor to the drift, independent of the reward-scaling problem, and it should be fixed — but it changes the objective, so every result in this section would need regenerating.

Algorithm audit

A line-by-line read of `grpo.py` against the GRPO/DAPO specifications, prompted by the E17 divergence. Recorded in full because two of the findings change the objective, and any result in this section predating a fix has to be read in that light.

Verified correct. The reference-KL estimator is the standard low-variance form $\hat{k}_3 = \rho - \log \rho - 1$ with $\rho = \pi_{\text{ref}}/\pi_\theta$, non-negative and applied with the right sign. π_{old} is stored from the rollout and the update is teacher-forced through the same decoding order and sampled actions, so the importance ratio is exactly 1 on the first inner update, as it must be. The encoder is deliberately frozen — only `decoder_layers` and `W_out` receive gradient — and computing the encoding under `no_grad` is consistent with that rather than an omission. Asymmetric clipping follows DAPO.

Finding 1: the trust region is applied per sequence, not per token. The surrogate computes $\min(\sum_t \rho_t A, \sum_t \text{clip}(\rho_t) A)$ where PPO and GRPO specify $\sum_t \min(\rho_t A, \text{clip}(\rho_t) A)$. A minimum of sums is not a sum of minima: the implemented form takes one clip/no-clip decision for an entire $L = 180$ token sequence, so a single token past the boundary switches every other token onto the clipped branch and zeroes its gradient. On $\rho = (5.0, 0.1)$ with $A > 0$ the correct objective is 1.38 with gradient $(0, 0.1)$; the implemented one gives 2.08 with gradient $(0, 0)$. With 180 tokens some token triggers the clip nearly always.

Finding 2: the advantage is normalised over the wrong group. GRPO’s defining feature is that the baseline is the mean reward over the group sampled from one prompt. Here the advantage is recomputed inside the inner loop on each randomly drawn sub-batch of `update_batch_size` = 8 out of $B = 16$, rather than once over the full rollout. The same sequence therefore receives a different advantage on each of the 16 inner updates, and sequences near the median receive advantages of *both signs* — 3 of 16 in a representative draw — so the update pushes the same rollout up and down within one step. The fix is to compute the advantage once, before the loop.

Finding 3: no length normalisation. $\sum_t \rho_t$ scales with sequence length, and the sum runs over all L positions including residues held fixed by `fixed_residues`, which contribute $\rho \approx 1$ each. The effective step size therefore depends on how long the protein is and how much of it is designable. The standard form is a mean over designed tokens.

Deeper pass: the rest of the loop. A second audit covering the rollout, the training loop and checkpointing found the following.

Clean. Dropout is constructed at $p = 0$ throughout, so `model.train()` injects no stochasticity and the importance ratio is genuinely 1 on the first inner update rather than only nominally so. Sampling probabilities are taken from `logits.detach()`, correctly keeping the sampling step out of the gradient. Row counts in the per-step metrics are a constant 16 across all 275 logged steps

of the two completed runs, so no fold failure has silently dropped a design; were one to, the inner merge would shorten the reward vector and the indexing would raise rather than misalign.

Finding 4 (now fixed): the behaviour policy is not the policy being scored. Actions are drawn from $\text{softmax}((\ell + b)/T)$ renormalised over the 20 residue tokens, where $b = -10^8$ at amino acids excluded by `omit_AA`. The log-probabilities used for the ratio are $\log \text{softmax}(\ell)$ over all 21 tokens — no bias, no temperature. The two differ by $\log P(\text{allowed})$, which depends on θ and therefore does not cancel between π_{new} and π_{old} : the true ratio is the implemented one times $P_{\text{old}}(\text{allowed})/P_{\text{new}}(\text{allowed})$. While the mass the model places on C and X stays put this factor is 1 and nothing is wrong; if the policy drifts mass onto the omitted tokens — which nothing in the objective discourages, since they are merely never sampled — the error grows with the drift and is not currently monitored. A policy putting 2% on omitted tokens where it once put 1% carries a 1% systematic error in every ratio.

The same mismatch makes **temperature** a trap rather than a knob. At $T = 1$ the scored and sampled log-probabilities differ by ~ 0.05 nats; at $T = 0.5$ by 1.3; at $T = 0.1$ by 17.8. Training at any temperature other than 1 silently optimises a different distribution from the one it samples. `CLAUDE.md` already advises keeping $T = 1$ to avoid mode collapse; this is a second and stronger reason.

Scoring now uses $\log \text{softmax}(((\ell + b)/T)_{1:20})$, padded to 21 channels — exactly the distribution actions are drawn from, verified equal to the true sampling log-probability to float precision at $T \in \{1, 0.5, 0.1\}$ where the previous form was off by 0.17, 4.3 and 44.2 nats respectively. The pre-fix behaviour is retained behind `legacy_logprobs` for the ablation. Separately, every run now logs `p_omit`, the probability mass the policy places on tokens `omit_AA` forbids, since that is precisely the quantity whose drift makes the omitted correction bite and nothing in the objective holds it down.

Finding 5: the final policy was never saved. `_last.pt` was written only inside the “every n steps” branch, so a 75-step run left it at step 50 and a 200-step run would leave it at 175. Confirmed directly from the E15 artefacts: the four checkpoints on disk are steps 25, 50, `_best` (25) and `_last` (50), for a run that trained to step 74. Under rank normalisation those runs were still improving when they ended, so the discarded tail was the best part of the run.

Finding 6: `_best.pt` selects on a constant. Best-checkpoint selection compared to `_log["reward"]`, which under a rank-normalised single-metric reward is exactly 0.5 at every step. With a strict $>$ test the first checkpoint written wins permanently — which is why every E15 arm reports its best at step 25 or 50 and never later. **Both are fixed:** `_last.pt` is now written after the loop at the true final step, and selection runs on a configurable `checkpoint_metric` (`ame_motif_rmsd_batch_mean`, mode `min`) with a warning emitted if the tracked quantity never varies.

All three are now fixed, and the pre-fix objective is retained behind `legacy_surrogate: true` so the fix can be measured rather than merely asserted. The surrogate takes a per-token minimum and averages it over designed tokens only — `chain_mask` is zero at fixed residues, whose log-probs the rollout already zeroes, so including them would contribute $\rho = 1$ apiece and make the step size depend on how much of the protein is held fixed, which disposes of Finding 3 as well. The advantage is computed once over the full rollout group before the update loop.

The A/B ladder. Four 200-step arms on M0097, each turning on exactly one more fix than the last, so any effect can be attributed:

Arm	Surrogate	Log-probs	KL
<code>conv200_legacy</code>	legacy	legacy	—
<code>conv200_surr</code>	fixed	legacy	—
<code>conv200</code>	fixed	fixed	—
<code>conv200_kl</code>	fixed	fixed	0.02

All four log `p_omit` and all four checkpoint on `ame_motif_rmsd_batch_mean`. The prediction under

test: if the entropy climb is driven by the blunt trust region and the noisy baseline rather than by reward scaling alone, `conv200_legacy` should reproduce it and the corrected arms should not. Figure 1 is the panel these land in — it discovers arms on disk, so it fills out as they finish.

The KL gating defect noted under E17 *has* been fixed — a non-zero `kl_weight` now implies the reference model — since that one only ever silenced a penalty the config asked for.

E16 result — selecting for diversity during training also fails

The prospective version of E9. Two 75-step arms on M0097, identical but for the rule used to pick which 16 of an oversampled pool of 64 get folded: uniform random, or greedy farthest-point (`maxmin`). Best-of-5 folding, so pass rates here are on the benchmark’s unit and are not comparable to the single-sample E15 numbers.

Rule	Pass rate	Passing	Spread	U	U_{50}
<code>random</code>	36.7 %	440	93.3	2423	1041
<code>maxmin</code>	4.6 %	55	121.6	1545	1486

The rule does exactly what it is designed to do and the design is still wrong. Per passing design, `maxmin` genuinely explores more mutational space — U_{50} of 1486 against 1041, a factor of 1.43 at matched design count, and a spread of 121.6 against 93.3. It simply produces *eight times fewer* such designs, so the library actually obtained covers 1545 distinct substitutions against random’s 2423. Diversity per design is up $1.43\times$; total library coverage is down $1.57\times$.

This is the same conclusion as E9 and a stronger version of it. E9 selected retrospectively from a finished pool, where the only cost was picking sequences that happened to fail. Selecting during training adds a second cost on top: because the selection depends on the sampled actions, the batch GRPO computes its group-relative baseline over is no longer a sample from the policy, so the importance ratios apply to a distribution that was never sampled — the gradient is biased by construction, as the docstring on `collect_rollout` warns. The 36.7 % versus 4.6 % gap is far too large to be selection cost alone.

Consequence. Both the retrospective and the prospective experiment now say the same thing: deciding what to fold should be left to the policy. The defensible use of these rules remains post-hoc, choosing an orderable subset from designs already known to pass.

Re-measured without selection, the result stands. The table above is a pass rate over *folded* designs, which E18 later showed cannot be compared across selection rules — so the conclusion was re-derived on the one measurement that can be. Both checkpoints were sampled 96 times with no selection applied and folded best-of-5.

Rule	Pass rate	Median RMSD	U	U_{15}	Spread
<code>random</code>	77.1 %	1.24 Å	1133	477	82.8
<code>maxmin</code>	19.8 %	1.88 Å	867	768	107.0

Every direction survives and the effect is not marginal: 19.8 % against 77.1 % ($p = 1 \times 10^{-15}$), with median motif RMSD pushed from 1.24 Å to 1.88 Å — from inside the 1.5 Å cutoff to well outside it. Per-design coverage is up $1.61\times$ and total library coverage is down to $0.77\times$, the same trade the folded-set numbers reported at larger magnitude. `maxmin` at oversample 4 damages the policy itself, not merely the set chosen from it.

This is worth stating precisely because the same rule is *harmless* in E18 at oversample 8, where it is indistinguishable from its control (70.8 % against 68.8 %). The two runs differ in both pool size and step

count — 75 steps here, 150 there — so this pair cannot attribute the reversal to either one, and the E18 prediction that a larger pool would make `maxmin` worse is contradicted rather than confirmed. Note also that the `random` controls differ, 77.1 % at step 74 against 68.8 % at step 149, which is the decay already documented at `kl_weight = 0` and a reminder that these two experiments are not on a common baseline.

E18 — was E16’s negative about selection, or about selecting on distance?

Pre-registered. Six 150-step arms on M0097, identical but for the rule used to pick which 16 of an oversampled pool of 128 get folded. Predictions are recorded here *before* the runs finish so the result can falsify them rather than be read post hoc.

E16 established that picking designs to fold by farthest-point traversal is much worse than picking them at random — 4.6 % passing against 36.7 %. But it lost for a diagnosable reason: `maxmin` is drawn to sequences that are distant *because they are degenerate*. On a synthetic pool built with a degenerate low-probability tail, 94 % of its picks come from that tail. That diagnosis implicates the axis, not the idea: every rule available to E16 scored on sequence distance alone, and the policy’s own log-probability of each sampled sequence — computed during the rollout, free to reuse — is a quality signal none of them could see.

Rule	Axis	Prediction
<code>random</code>	control	The bar. At matched oversample, so the arm separates “this rule helped” from “oversampling helped”
<code>logprob_strat</code>	quality	Best, or the only rule to beat random. Targets reward spread <i>within the group</i> , which is what the group-relative baseline divides by
<code>logprob_top</code>	quality	Folds excellent designs and stalls anyway, because the group’s reward spread collapses. If it trains well, the group-baseline story is wrong
<code>logprob_band</code>	hybrid	The targeted repair of E16: same diversity objective, degenerate sequences removed from the pool first
<code>maxmin</code>	distance	Loses again, harder — a larger pool has a more extreme tail to walk to
<code>cluster_rep</code>	distance	Also loses. On synthetic pools it takes 94 % of its picks from the degenerate tail too, so “bounded” was never the operative property

The bias is instrumented rather than corrected. As in E16 the selected batch is not a draw from the policy, so the importance ratios apply to a distribution that was never sampled. Two diagnostics now record what that costs. `sel_logp_shift` is how far the rule moved the batch in the policy’s own measure — the pool’s rewards are never observed, since only the 16 are folded, so a selected-versus-pool *reward* gap is not computable and this is the honest substitute. `reward_std` is the spread the group-relative advantage actually divides by, and it is the test of whether a rule bought the thing it was chosen for, independently of whether the run ends up better.

Cost note: folding is 16 designs $\times$ 5 diffusion samples per step regardless of the oversample factor, so runtime scales with `N_steps` and not with the pool. Raising 64 to 128 is close to free.

E18 — two design errors, caught mid-run

Both are recorded rather than quietly repaired, because one of them invalidates a rationale stated above and the other would have produced a headline number that is pure artefact.

1. reward_std is degenerate under rank normalisation. It was added as the test of whether a rule bought the reward spread it was chosen for. It cannot be: with `normalize: rank` the reward is the within-batch rank mapped to $[0, 1]$, so its standard deviation is fixed by the batch size alone. Measured, it is 0.317397 in *every arm at every step* — two or three distinct values across an entire run, all from tie-averaging — against a theoretical 0.3073 for 16 evenly spaced ranks.

This is not merely a dead diagnostic. The stated rationale for `logprob_strat` was that it targets reward spread within the group, which is what the group-relative baseline divides by. Under rank normalisation that spread is *already* constant, so the rule cannot be operating through the mechanism it was designed around. Whatever it is doing, it is not that. Testing the mechanism as written would require an arm with value normalisation, where the spread is free to vary.

2. The pass rate over folded designs cannot be compared across arms. It is the obvious summary and it is confounded by construction: each rule chooses which 16 of 128 get folded, so a rule that selects for quality reports a high pass rate whether or not its policy improved at all. `logprob_top` selects the 16 most probable sequences and reports 78.3% against `random`’s 25.2%; almost all of that gap is the selection, not the policy.

This is the same trap as the `as-sampled` row in E9, running the other way. There it was a policy-quality effect masquerading as a selection effect; here it is a selection effect masquerading as policy quality. Entropy over folded designs is confounded for the same reason — `maxmin` selects for diversity, so of course its folded set is diverse.

Consequence. E18 cannot be read off the training logs at all. The comparison has to be made by sampling from each final checkpoint *without selection* and folding that, which is the only measurement in which the arms differ solely by the policy they produced. The training-log columns retained below are the ones that are not confounded: `sel_logp_shift`, which is a property of the rule rather than of the outcome, and which confirms every rule does what it claims — 29.7 nats for `logprob_top`, −19.2 and −18.3 for `maxmin` and `cluster_rep`, and −0.04 and −0.34 for the stratified and banded rules, which move the batch almost not at all in the policy’s own measure.

E18 result — the frontier is real, and the mechanism we pre-registered is not the one operating

Measured as the previous subsection requires: 96 samples drawn from each arm’s final checkpoint *without selection*, folded best-of-5. One shared seed across arms, so the comparison is paired and the arms differ only by their weights. U and U_{15} are over passing designs, counted against one shared reference so the arms sit on a single scale; every passing design in every arm was unique, so deduplication is a no-op here.

Rule	shift	Pass rate	p	U	U_{15}
random	−0.4	68.8 %	—	1237	558
logprob_strat	0.0	87.5 %	0.003	1141	525
logprob_top	29.7	82.3 %	0.043	1304	603
logprob_band	−0.3	65.6 %	0.759	1354	707
maxmin	−19.2	70.8 %	0.875	1298	641
cluster_rep	−18.3	56.2 %	0.101	1349	746

Superseded by E19 — read the table above as one draw, not as a result. E19 re-ran this same config on this same backbone in a fresh process and did not reproduce it. `logprob_top` fell from

82.3% to 46.9% ($p = 4 \times 10^{-7}$) and the `random` control moved *up*, 68.8% to 89.6% ($p = 6 \times 10^{-4}$), so the headline gap reverses sign between two draws of the same experiment. The pre-registration for E19 named this outcome in advance: “if M0097 itself does not reproduce, E18 was noise.” The paragraphs below are kept as written because the pre-registered reasoning they falsify is still worth recording, but every pass-rate claim in them is retracted. The coverage ordering does replicate; see the E19 result.

The pre-registered mechanism is falsified, by the test written down for it. `logprob_top` was predicted to fold excellent designs and stall anyway, with the explicit rider that if it trained well the group-baseline story was wrong. It trains well: it is the one arm that beats `random` on *both* axes, 82.3% against 68.8% ($p = 0.043$) at U_{15} 603 against 558. Pure exploitation, the rule with the least reward spread available to it, is not merely survivable but dominant over the control. This is the same conclusion the `reward_std` defect reached from the other direction: under rank normalisation the group’s spread is constant by construction, so no rule could have been operating through it.

`logprob_strat`, predicted best, is half right in a way that matters. It does take the highest pass rate, 87.5% ($p = 0.003$), and it is the largest single effect in the arm. But it is the *only* rule whose coverage falls below the control on both measures — U 1141 against 1237, U_{15} 525 against 558. It bought its pass rate by collapsing the policy, which is precisely the trade the whole method exists to refuse. It was also not the only rule to beat random, the other half of the prediction.

The two distance rules did not lose. `maxmin` was predicted to lose again and harder; it is indistinguishable from the control on pass rate (70.8%, $p = 0.88$) while carrying higher coverage. `cluster_rep` takes the lowest pass rate, 56.2%, but the gap does not reach significance ($p = 0.10$) and it returns the highest per-design coverage in the arm, U_{15} 746 — a factor of 1.34 over the control. Neither is E16’s eight-fold catastrophe. They trade along a frontier rather than falling off it.

`logprob_band` is the rule that worked as designed, and the diagnostic shows the mechanism directly. It was built as the targeted repair of E16: keep the diversity objective, remove the degenerate tail it was drawn to. Its `sel_logp_shift` is -0.3 , so it moves the batch essentially not at all in the policy’s own measure — but its selected log-probability spread is 8.7 against the pool’s 18.2, so it is exploring inside a truncated band rather than walking to the edge. The outcome matches: U_{15} up 27% over the control at no pass-rate cost that the data can resolve ($p = 0.76$).

What survives. Selection at $128 \rightarrow 16$ is not the dead end E16 made it look. The control is not on the frontier — `logprob_top` dominates it outright, and `logprob_band` buys a quarter more coverage for a pass-rate difference the experiment cannot resolve. What is dead is the reason we expected it to work.

What this cannot support. One run per arm. The binomial interval on a pass rate at $n = 96$ is roughly ± 9 points before seed variance is added, and the A/B replicates put seed variance on top of that — so the three arms with $p > 0.1$ are genuinely unresolved, not tied. The coverage ordering spans $1.42\times$ (525 to 746), comfortably outside the bootstrap noise scale of ± 25 –40; note that a bootstrap of a distinct-count statistic is biased *downward*, since resampling with replacement introduces duplicates that can only reduce U , so those intervals are read for width and not for position. Replicating `logprob_band` and `logprob_top` at $n = 3$ is the cheap next step and the one that would license a recommendation.

The open question this reopens. E16 concluded that `maxmin` damages the policy. That conclusion rests on a pass rate over folded designs — the exact quantity error 2 above shows cannot be compared across selection rules. Both E16 checkpoints survive, so the same selection-free harness settles it directly; that evaluation is running. If E16’s `maxmin` policy is fine and only its folded set was bad, E16’s headline needs rewriting to a claim about wasted folding compute rather than about policy damage.

E19 — does the frontier belong to the rules, or to M0097?

Pre-registered. Four rules — `random`, `logprob_top`, `logprob_band`, `logprob_strat` — on three backbones, 150 steps, oversample 8, evaluated by the same selection-free harness E18 required. The two distance rules are dropped: neither separated from the control on pass rate, and E16’s re-evaluation covers `maxmin` directly.

Nothing seeds the trainer explicitly, so re-running M0097 in a fresh process is an independent draw rather than a repeat. That makes the M0097 arm a seed replicate of E18 and the M0904 and M0907 arms a test of generalisation, and the two failure modes are distinguishable: if M0097 reproduces and the other two do not, the frontier is a property of that backbone; if M0097 itself does not reproduce, E18 was noise.

Predictions. `logprob_top` continues to dominate `random` on both axes — this is E18’s one unambiguous positive and the claim most worth breaking. `logprob_band` continues to buy coverage at no resolvable pass cost. `logprob_strat` continues to raise pass rate while falling below the control on coverage; it is included precisely because a collapse result is the counterweight to the headline, and a headline whose counterweight does not replicate is not worth reporting either.

Three backbones give three paired contrasts per rule rather than three draws of one number, which is the stronger design — but it is still $n = 1$ per cell. The arm can show that a direction is consistent across backbones; it cannot put a tight interval on the size of the effect on any one of them.

E19 result — the frontier does not replicate, and the reason is seed variance

Twelve runs, 150 steps, oversample 8; 96 selection-free samples per arm from the final checkpoint, folded best-of-5. Same harness as E18. p is Fisher’s exact against that backbone’s own `random` control.

Backbone	Rule	Pass	p	Med. RMSD	U	U_{15}
M0097	<code>random</code>	89.6 %	—	1.04 Å	1022	432
	<code>logprob_strat</code>	82.3 %	0.21	1.15 Å	1208	579
	<code>logprob_top</code>	46.9 %	2×10^{-10}	1.56 Å	1362	787
	<code>logprob_band</code>	64.6 %	6×10^{-5}	1.42 Å	1403	721
M0904	<code>random</code>	63.5 %	—	1.41 Å	709	333
	<code>logprob_strat</code>	32.3 %	2×10^{-5}	1.68 Å	559	413
	<code>logprob_top</code>	33.3 %	5×10^{-5}	1.68 Å	803	567
	<code>logprob_band</code>	9.4 %	2×10^{-15}	2.63 Å	580	580
M0907	<code>random</code>	54.2 %	—	1.48 Å	614	323
	<code>logprob_strat</code>	20.8 %	3×10^{-6}	1.74 Å	484	425
	<code>logprob_top</code>	50.0 %	0.66	1.49 Å	859	488
	<code>logprob_band</code>	0.0 %	3×10^{-20}	2.07 Å	0	0

No rule beats the control on pass rate on any backbone. The best result any rule achieves against its own control is a tie (`logprob_top` on M0907, 50.0 % vs 54.2 %, $p = 0.66$). Pooled over the three backbones at $n = 288$ per arm, `random` takes 69.1 % against 45.1 % for `logprob_strat`, 43.4 % for `logprob_top` and 24.7 % for `logprob_band`, every one of them a loss at $p < 1 \times 10^{-8}$. E18’s central claim — that `logprob_top` dominates the control outright — does not survive.

It fails first on E18’s own backbone, which localises the problem to seed variance rather than to backbone specificity. The M0097 arm is a straight re-run of the E18 config in a fresh process; nothing seeds the trainer, so it is an independent draw. `logprob_top` went 82.3 % $\rightarrow$ 46.9 % ($p = 4 \times 10^{-7}$). The *control* went 68.8 % $\rightarrow$ 89.6 % ($p = 6 \times 10^{-4}$). Two draws of one configuration differ by 21 points on the control alone — larger than almost every effect E18 reported,

and in the opposite direction from the one that mattered. This is the branch the pre-registration named: E18 was noise.

This generalises past E18 and we should say so plainly. A ± 21 -point seed swing on a pass rate means any single-run pass-rate contrast in this paper at $n \approx 96$ is under-powered, not merely imprecise. The binomial interval we quoted for E18 (± 9 points) described sampling error inside one run and silently omitted run-to-run variance, which is the larger term. Every $n = 1$ pass-rate comparison in the slate inherits this, and the fix is replication, not a wider interval on one draw.

What does replicate is the coverage trade. `logprob_top` returns more distinct substitutions per design than the control on five of the six backbones tested across E19 and E20, at U_{15} ratios of 1.82, 1.70, 1.51, 2.07 and 2.68 (mean 1.79); the sixth is a wash at 0.98. That direction is far steadier than anything in the pass-rate column. So there is a real frontier here, but it is a *trade* — coverage bought with pass rate — and not the dominance E18 claimed. `logprob_strat` is likewise consistent, and consistently mediocre: coverage up 1.24 to $1.34\times$ on all three E19 backbones, pass rate down on all three.

logprob_band is the clearest casualty. E18 read it as the rule that worked as designed. Across three fresh backbones it takes 64.6 %, 9.4 % and 0.0 % — on M0907 not a single one of 96 designs passes, with median motif RMSD at 2.07 Å against a 1.5 Å cutoff. Its sequence-level diversity was the most stable quantity we measured, 102 to 107 mean pairwise Hamming on all six backbones, and it is precisely that stability that turns out to be the warning: the band holds the policy at a fixed spread in its own log-probability measure whether or not that spread is anywhere near the motif. Diversity decoupled from the objective is not exploration, and the interim sequence-level table we reported before folding pointed the wrong way for exactly this reason.

E20 — can selection rescue a backbone the benchmark scores 0/40?

Pre-registered. `random`, `logprob_band` and `logprob_top` on M0050, M0365 and M0375 — the three near-misses from E3, all scored 0/40 by the published benchmark. The matched `random` control at the same oversample is the comparator, not E3’s numbers, which differ in both step count and oversample.

This is a different regime from E18 and the distinction is the whole point of running it. E18 measured these rules where 56 % to 88 % of designs already pass; these backbones sit near 0.1 % to 0.5 %, with median motif RMSD of 2.8 Å to 3.1 Å against a 1.5 Å cutoff and every E3 hit an excursion into the tail rather than a shifted mode.

The mechanistic reason to expect little. A selection rule chooses among the designs the policy already produced. If nothing in the pool is close, no choice among them manufactures a hit — selection re-ranks, it does not generate. On a backbone whose median sits 1.4 Å outside the cutoff, the pool is mostly not close.

The reason to run it anyway. The quantity that has to move is the median, not the pass rate, and `logprob_band`’s effect is on coverage: it keeps the policy exploring distinct regions instead of walking to the degenerate edge, which is more places to find a mode. So the pre-registered prediction is a split one — *pass rate does not move on any of the three, and if anything moves it is median RMSD under logprob_band*. Pass rate is close to useless as a readout here in any case: at these rates a 150-step run returns single-digit counts, so median and minimum motif RMSD are the primary endpoints and the pass count is reported only for continuity with E3.

A null here is publishable and worth stating in advance as such: it would establish that the E18 frontier is a property of the high-pass-rate regime and does not transfer to hard targets, which bounds the claim rather than weakening it. The failure mode to guard against is reading a one- or two-design difference in a 1200-fold pool as a rescue — E3’s replication at $N = 40$ already showed that four of its seven hits do not survive refolding.

E20 result — selection does not rescue, and the training already did

Nine runs, all nine folded. Pass counts are single digits on two of the three backbones, so median RMSD is the endpoint to read, exactly as pre-registered.

Backbone	Rule	Pass	p	Med. RMSD	U_{15}
M0050	random	9.4 %	—	1.62 Å	239
	logprob_top	8.3 %	1.0	1.83 Å	495
	logprob_band	0.0 %	0.003	3.34 Å	0
M0365	random	1.0 %	—	2.18 Å	50
	logprob_top	1.0 %	1.0	2.16 Å	134
	logprob_band	0.0 %	1.0	2.63 Å	0
M0375	random	57.3 %	—	1.44 Å	420
	logprob_top	6.2 %	6×10^{-15}	2.15 Å	412
	logprob_band	0.0 %	8×10^{-22}	2.39 Å	0

The pre-registered prediction holds in full. Pass rate does not move on any backbone: two exact ties and two large losses, both on M0375. The one place we said movement might appear — median RMSD under `logprob_band` — moved the wrong way on all three, 2.4 Å to 3.3 Å against controls at 1.4 Å to 2.2 Å, and `logprob_band` passes nothing at all: 0/96 on every backbone, 0/288 in total. Selection re-ranks; it does not generate. On a pool that mostly is not close, that is the whole story.

The unplanned result is more interesting than the planned one. These three backbones were selected as benchmark failures scored 0/40, and the `random` control — which is CLEO training with no selection rule at all — passes 57.3 % on M0375 and 9.4 % on M0050, with median motif RMSD inside the cutoff on both. The rescue is real and it is already done by the RL fine-tuning, independent of anything in this experiment. That is E3’s claim reproducing on an unconfounded harness, which is a stronger form of it than E3 itself reported.

It also means the framing in this experiment’s pre-registration — these backbones “sit near 0.1 % to 0.5 %” — describes the *published benchmark’s* designs, not a trained policy’s, and should not be read as the comparator for the table above. M0365 is the one that stays hard: 1/96 for both arms, median 2.2 Å. One backbone out of three resisting training is the honest headline for the rescue claim, and it is a claim about training, not about selection.

E3 result — two backbones reach the cutoff, one convincingly

M0050_1dbt_cond2_6, one of the three near-miss backbones the published benchmark scores at 0/40 with a best-of-40 motif RMSD of 1.61 Å to 1.69 Å. 75 steps, batch 16, best-of-5.

Backbone	Published	Passing	Best RMSD	Margin	Median
M0050_1dbt	0/40, 1.61 Å	1 / 1200	1.477 Å	0.023 Å	2.81 Å
M0365_1pfl	0/40, 1.69 Å	6 / 1200	1.273 Å	0.227 Å	3.12 Å
M0375_4ts9	0/40, —	2 / 1200	1.493 Å	0.007 Å	3.00 Å

The three differ by more than their counts suggest. M0050 contributes a single design 0.023 Å inside the cutoff — one draw from a distribution sitting on the threshold. M0365 contributes six

distinct sequences, the best at 1.273 Å, a margin ten times larger and well clear of the prediction noise that makes M0050 unreadable. M0375 contributes two designs at 1.4926 Å and 1.4968 Å — margins of 0.007 Å and 0.003 Å, thinner even than M0050’s. Both fail replication outright (below). Of the three backbones only M0365 carries a claim.

On all three targets the clash term is never binding (100 % clash-free throughout), so the composite criterion reduces to motif RMSD alone and the no-clash half of the reward contributes nothing on these backbones.

There is a pattern across the three: median RMSD is 2.8 Å to 3.1 Å everywhere, far from the cutoff, and every hit sits in the extreme tail of its own run’s distribution. These are hard targets on which the policy is not close in general, and the hits are excursions rather than a shifted mode — which is exactly the regime in which a best-of- N filter selects tail draws, and exactly what the replication below shows.

Replication at $N = 40$ settles it, mostly against us. Every one of the seven hits was refolded 40 times — the published per-backbone budget — and the pass fraction counted.

Design	Backbone	Passing / 40	Min RMSD	Median RMSD
hit00	M0365	22	1.196 Å	1.470 Å
hit01	M0365	2	1.410 Å	1.818 Å
hit04	M0365	1	1.454 Å	2.327 Å
hit05	M0050	1	1.456 Å	1.920 Å
hit03	M0365	0	1.514 Å	1.926 Å
hit06	M0365	0	1.638 Å	2.019 Å
hit02	M0365	0	1.729 Å	2.430 Å

M0375’s two hits were checked the same way and pass **0/40** each, with best refold RMSDs of 1.577 Å and 1.658 Å — both above the cutoff. Across all three backbones, nine designs cleared the criterion during training and **one** reproduces robustly. **hit00** passes 22 of 40 predictions with a *median* RMSD of 1.470 Å — below the cutoff, not merely touching it — on a backbone the published benchmark scores 0/40 with a best of 1.69 Å. That is a real result and the one worth reporting.

Five of the nine pass *zero* of 40 on refolding. M0050’s single hit passes 1/40, exactly the tail draw its 0.023 Å margin predicted; neither the M0050 nor the M0375 rescue survives and both should be dropped. Margin at training time predicts replication well: the only design with a margin above 0.03 Å is the only one that reproduces.

The general lesson matters more than the rescue. These seven were selected *because* they passed, so they are the winners of a noisy filter and regression on refolding is expected — but the size of it is the point. A design passing 1/40 of the time is flagged by a best-of-5 training filter about 12 % of the time it is folded, so a best-of-5 label is a weak indicator of reproducible success, and the in-training pass rates reported throughout this section (83 % for the corrected arm, 57 % for the anchored one) are counts of best-of-5 events rather than of designs that would survive this test.

They are not *wrong* — best-of- N is the benchmark’s own definition, and by that definition four of these seven clear it where the published run cleared none. But the two quantities should not be conflated, and any headline number that leaves this section ought to be replicated the way **hit00** was. The cheap version is to raise `num_diffusion_samples` at evaluation time only; the honest version is to report both.

Still open: the compute is not matched (E4), and the idealised-versus-unidealised reference-structure question still blocks M0904 and M0907.

A/B result — the fixes work, and the failure mode inverts

Four 200-step arms on M0097, each one fix further than the last, all with rank normalisation and best-of-5 folding.

Arm	Success	Entropy	Spread	U	U_{15}	succ.	div.
legacy objective	78.0 %	0.88	69.8	1879	467	$1.71\times$	$1.21\times$
+ surrogate fix	88.8 %	1.09	82.6	2286	575	$2.65\times$	$1.43\times$
+ log-prob fix	83.1 %	0.99	76.6	2109	510	$2.16\times$	$1.32\times$
+ KL anchor	56.7 %	1.65	114.4	2501	808	$3.98\times$	$1.93\times$

All four arms completed 200 steps; statistics are over their final 40 (640 designs each), with ratios against the $n = 96$ baseline frontier interpolated on its monotone branch ($T \geq 0.3$).

The divergence is gone. Every corrected arm converges: success climbs monotonically and stays climbing, where the original run peaked at 13.8% near step 40 and decayed to zero. Even the `legacy_surrogate` arm converges, because it still carries rank normalisation — which confirms the E17 diagnosis that the reward’s compressed dynamic range, not the surrogate, was the primary cause of the collapse. The surrogate and advantage bugs cost real performance on top of it: 78.0% to 88.8% success, and a $1.71\times$ to $2.65\times$ jump in the frontier ratio.

That last comparison was held back pending replication, and replication killed it. Training is unseeded — no `manual_seed` anywhere in the pipeline — so every run is a different trajectory, and the table above has $n = 1$ per arm. Two further replicates of each of the two contested arms give $n = 3$ on exactly that comparison:

Arm ($n = 3$)	Success	Entropy	U	U_{15}
+ surrogate fix (no log-prob fix)	86.0 ± 4.7	1.06 ± 0.04	2225 ± 57	544 ± 31
+ log-prob fix	88.3 ± 4.9	0.92 ± 0.10	2074 ± 61	485 ± 53
Welch p	0.60	0.13	0.036	0.19

The 88.8% to 83.1% gap was seed noise, and it does not merely shrink — it *reverses* sign, to 88.3% against 86.0%, at $p = 0.60$. Within-arm spread is ± 5 points, which is larger than the effect it was being asked to support. Refusing to report that difference was correct.

What survives is weaker and points the other way: the log-prob fix costs total coverage, U 2225 to 2074 at a nominal $p = 0.036$. That is one of four metrics tested and does not survive a multiplicity correction, so it is a hint rather than a result — but every diversity column leans the same direction and none of them favours the fix.

The log-prob fix is nonetheless kept, because it is a correctness fix and not a performance one. It makes the scored distribution match the distribution actually sampled from; without it the ratio in the surrogate is not a likelihood ratio between the two policies, so the clipping bounds do not mean what PPO/DAPO says they mean and the objective is not the one we claim to optimise. A known-wrong gradient is not made right by scoring two points higher on one metric, and given the measured seed variance those two points are not even evidence. This is worth stating plainly because it is the one place in the audit where the corrected code does not beat the buggy code on the headline number.

Whether the coverage cost is real needs $n \gtrsim 8$ per arm to resolve, which is not currently worth the GPU time: the KL anchor moves U_{15} by 300, and this effect, if it exists at all, is 60.

But the failure mode inverts. Entropy no longer rises; it falls, from ~ 1.7 to 1.06 in the fully corrected arm. Having fixed a policy that diffused outward until it was useless, we now have

one that converges onto a narrow solution: 79.2% success at spread 80.1 and $U_{15} = 518$, which is *below* $T=0.7$'s 608. On per-design coverage the corrected policy without an anchor is worse than a high-temperature baseline. Crucially, this is true of *both* corrected arms and all six replicates — mean U_{15} is 544 and 485, both well under 608 — so it is a property of the fixed objective rather than of any one run. Sharpening the gradient does not rescue it: no version of the correct objective, anchored or not, explores enough on its own.

The KL anchor is the knob against it. At `kl_weight` = 0.02 entropy stays flat across the whole run (1.68 to 1.65) instead of collapsing, and the library is the best of any arm: $U_{15} = 782$, 1.51 times the unanchored arm and 1.29 times $T=0.7$, at spread 110.9. Over its final 40 steps the anchored arm reaches 56.7% success at spread 114.4 with $U_{15} = 808$ — **3.98×** the baseline success at matched diversity and **1.93×** the baseline diversity at matched success — against the unanchored arm's 83.1% at spread 76.6, $U_{15} = 510$ ($2.16\times / 1.32\times$). Entropy settles at 1.65 versus 0.99.

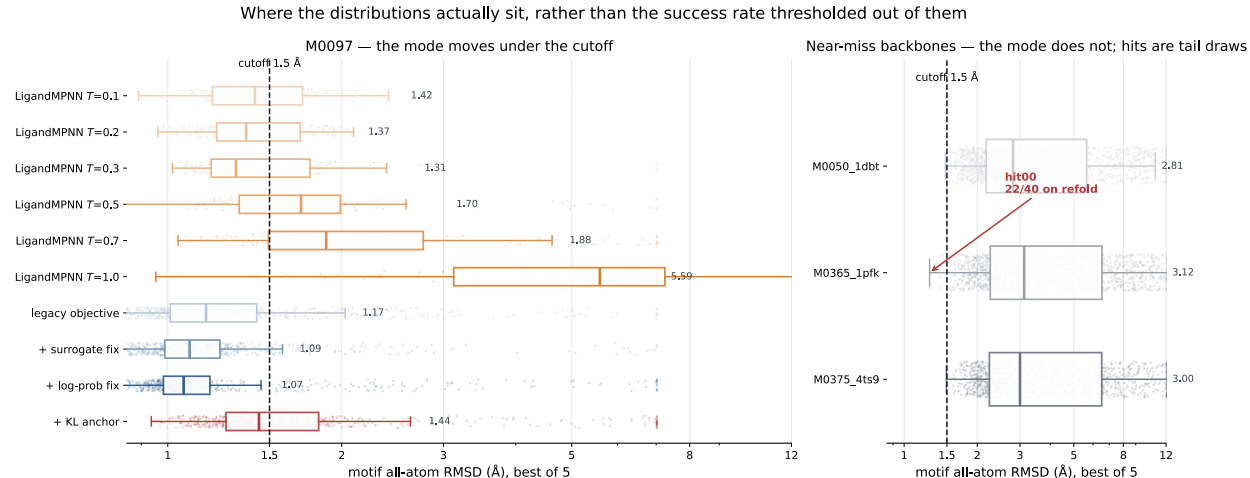


Figure 2: Where the distributions actually sit, rather than the success rate thresholded out of them. Each row is one arm’s per-design best-of-5 motif RMSD: box for median and quartiles, jittered strip behind it for the shape the box hides. Log axis, because a linear one lets the single worst arm compress everything that matters into the left fifth. **Left, M0097:** every CLEO arm has its *median* below the cutoff (1.07 Å to 1.44 Å) — the mode has moved, not just the tail — against 1.31 Å to 1.42 Å for low-temperature LigandMPNN and 5.59 Å at $T=1.0$. The non-monotonicity noted in E10 is visible directly: $T=0.3$ has a *lower* median than $T=0.1$. **Right, the three near-miss backbones:** medians of 2.81 Å to 3.12 Å with the cutoff far out in the left tail. The one design that survived 40 refolds sits at the extreme edge of its own distribution. Encoding: both ladders are ordinal, so each is a single hue ramped light to dark rather than arbitrary categorical colours; the anchored arm keeps an accent hue as the highlighted result.

This supersedes the E10 comparison, whose CLEO row came from the uncorrected objective and gave $1.40\times / 1.31\times$. It also repairs the concession forced by the E10 rarefaction: with an anchor, CLEO no longer merely passes more often at an equal exploration rate — $U_{15} = 808$ against $T=0.7$'s 608 is a genuine per-design coverage advantage, which the unanchored arm does not have.

What this changes about the paper. `kl_weight` is the diversity control the narrative needs, and it is better behaved than E15’s centering weight: it holds entropy open without requiring a reference sequence to centre on. The obvious next experiment is a sweep of it, which is E13’s slot. Two caveats. 42.5% against 79.2% at matched steps is a real cost, and only a sweep says whether 0.02 is near the right place on that trade-off. And the anchored arm is still improving at step 194, so its numbers are a lower bound rather than a converged result.

E13 result — the anchor has an interior optimum, and 0.02 is in it

Both caveats above are now settled. The anchored arm did keep improving: run to 200 steps it reaches 56.7% over its final 40, not the 42.5% measured mid-climb, so the cost of anchoring is roughly half what that snapshot implied. And the sweep answers where to put w .

Table 1: KL weight against the reference policy. All arms: M0097, 200 steps, identical otherwise; statistics over the final 40 steps (640 designs). Diversity is computed on *passing, deduplicated* designs only. U is distinct (position, residue) substitutions in that set; U_{15} rarefies it to 15 designs so libraries of different size are comparable.

w	Passing (%)	Median RMSD (Å)	Entropy	Passing seqs	U	U_{15}
0	83.1	1.07	0.99	532	2109	510
0.005	81.1	1.22	1.15	519	2230	562
0.01	57.7	1.45	1.54	369	2498	760
0.02	56.7	1.44	1.65	363	2501	808
0.05	40.5	1.63	1.78	259	2480	858
0.10	10.2	2.14	1.85	65	1743	881

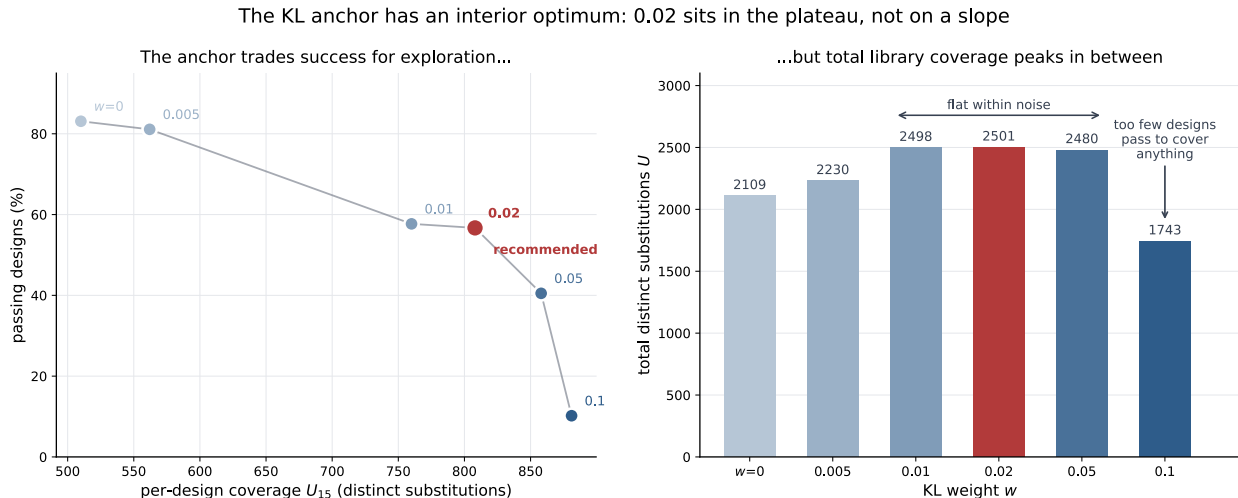


Figure 3: **The KL anchor has an interior optimum.** **Left:** the trade-off the anchor moves along — success rate against per-design coverage U_{15} , one point per weight. **Right:** total distinct substitutions U , the product of the two effects, which is flat across $w \in [0.01, 0.05]$ and collapses at $w = 0.1$. Drawn as two panels rather than one with two y -axes: success and U_{15} have unrelated scales, so on a dual axis the apparent crossing point could be placed anywhere by choosing the scales. Encoding: w is an ordinal ladder and gets one hue ramped light to dark; 0.02 takes the accent hue as the recommended default, not as another rung.

Two quantities move in opposite directions and it matters which one the library cares about. Success rate falls monotonically, 83% to 10%. Per-design coverage U_{15} rises monotonically, 510 to 881. Neither is the objective on its own: what a library needs is the *total* number of distinct substitutions it actually contains, and that is U , which is the product of the two effects and therefore **unimodal**. It is flat within noise across $w \in [0.01, 0.05]$ — 2498, 2501, 2480 — and falls off a cliff

at $w = 0.1$, where the anchor is strong enough that only 10 % of designs pass and there are too few of them to cover anything, despite each one being the most exploratory in the sweep.

So the answer to the question E17 left open is that 0.02 is in the plateau, not on a slope, and the choice within $[0.01, 0.05]$ is a genuine free parameter rather than a tuned one — it buys per-design coverage (760 to 858) with success rate (58 % to 41 %) at constant total yield. 0.02 is a defensible default because it sits in the middle of the flat region.

Read the $w=0.1$ arm as a failure, not as an endpoint. Its trajectory does not converge: binned pass rate runs 4.5, 5.0, 6.2, 7.8, 7.0, 11.5, 15.5, 6.5 — it is still wandering at step 200. The anchor is strong enough to prevent the policy from moving, so its high entropy is inherited from the reference rather than earned. That is the opposite failure from the unanchored arm’s collapse, and it is why U rather than entropy is the right summary: both extremes look good on exactly one of the two component metrics.

References